

Master Technical Playbook & Interview Guide

Guía de Arquitectura Distribuida, Clean Architecture, DDD, CQRS y Patrones en C# con Aplicación Práctica (Sistema Dental)

MÓDULO 01: ARQUITECTURA & PERFORMANCE EF CORE

01. Repository Pattern en EF Core: Mitos, Trade-offs y Realidad

El patrón **Repository** nació históricamente para desacoplar el acceso a datos directo (ADO.NET / SQL) de la lógica de negocio. Sin embargo, en .NET moderno, `DbSet<T>` ya implementa el Repository Pattern y `DbContext` actúa como Unit of Work.

El Anti-Patrón Generic Repository

Crear un `GenericRepository<T>` sobre EF Core fuerza una abstracción incompleta (*Leaky Abstraction*) que inhabilita las mejores capacidades del ORM:

- Destruye la ejecución SQL diferida (`IQueryable`) al forzar evaluaciones prematuras en RAM.
- Impide proyecciones directas a DTOs con `.Select()` generand queries pesados tipo `SELECT *`.
- Deshabilita optimizaciones críticas de memoria como `AsNoTracking()`.

Decisión de Arquitectura: En el lado de **Lectura (Queries)** no se usan repositorios; se consulta directamente `IQueryable` proyectando a DTOs. En el lado de **Escritura (Commands)**, se usan Repositorios Explícitos de Dominio únicamente para proteger los límites de los *Aggregate Roots*.

30-SECOND SENIOR ENGLISH PITCH

"Historically, Repositories decoupled business logic from data APIs like ADO.NET. In EF Core, `DbSet` is already a Repository and `DbContext` is a Unit of Work. Abstracting EF Core with a generic repository creates a leaky abstraction that hides `IQueryable` and hurts performance. Today, we query `IQueryable` directly with `AsNoTracking()` for Reads, and restrict custom repositories strictly to Aggregate Roots in DDD for Writes."

MÓDULO 01: ARQUITECTURA & PERFORMANCE EF CORE

02. Unit of Work Pattern y Gestión de Transacciones

El patrón **Unit of Work** mantiene un rastreo de los cambios acumulados durante una transacción de negocio para garantizar la atomicidad (ACID 'A'): todas las operaciones se guardan con éxito o ninguna se persiste en la base de datos.

Uso con EF Core y Transacciones Explícitas

En el dominio del **Sistema Dental**, al completar una cita se modifica el odontograma y se genera una factura de cobro. Si falla el registro financiero, la cita no debe marcarse como completada.

```
// Transacción atómica integrada en EF Core via Change Tracker
appointment.MarkAsCompleted();
var invoice = Invoice.Create(appointment.PatientId, appointment.Id, amount);
_context.Invoices.Add(invoice);

// DbContext ejecuta un único bloque de SQL Server Transaction
await _context.SaveChangesAsync(ct);
```

30-SECOND SENIOR ENGLISH PITCH

"DbContext natively implements the Unit of Work pattern through its Change Tracker. When SaveChangesAsync is called, EF Core wraps all pending Insert, Update, and Delete statements inside a single atomic database transaction. For coordinating external systems or stored procedures alongside EF Core, we use explicit transactions via IDbContextTransaction."

MÓDULO 02: DOMAIN-DRIVEN DESIGN (DDD)

03. Entities, Value Objects y Aggregate Roots (Dominio Dental)

Modelar un sistema clínico exige clasificar claramente los objetos del dominio para evitar modelos anémicos y garantizar la consistencia de las reglas de negocio.

Concepto	Identidad (Id)	Mutabilidad	Ejemplo Clínica Dental
Value Object	Sin Id. Definido por el valor de sus atributos.	Inmutable (C# <code>record</code>).	<code>ToothPosition</code> (Número de diente y cuadrante)
Entity	Id único constante en su ciclo de vida.	Mutable mediante comportamientos.	<code>ToothCondition</code> (Estado del diente en el odontograma)
Aggregate Root	Id global único. Controla acceso interno.	Mutable. Garantiza invariantes.	<code>Patient</code> (Raíz del agregado de Historia Clínica)

```
// Value Object
public record ToothPosition(int ToothNumber, Quadrant Quadrant);

// Aggregate Root que encapsula las entidades internas
public class Patient
{
    public Guid Id { get; private set; }
    private readonly List<ToothCondition> _odontogram = new();

    public void ApplyToothTreatment(int toothNumber, ToothStatus newStatus, decimal cost)
    {
        var tooth = _odontogram.FirstOrDefault(t => t.Position.ToothNumber == toothNumber);
        if (tooth == null || tooth.Status == ToothStatus.Missing)
            throw new DomainException("Invalid tooth for treatment.");

        tooth.ApplyTreatment(newStatus);
        RaiseDomainEvent(new ToothTreatmentAppliedDomainEvent(Id, toothNumber, cost, DateTime.UtcNow));
    }
}
```

30-SECOND SENIOR ENGLISH PITCH

"In DDD, Value Objects are immutable structures compared by value without an identity, like ToothPosition. Entities have a unique identity that persists across state changes, like ToothCondition. An Aggregate Root is the primary Entity—such as Patient—that acts as a gateway enforcing all business invariants for child entities."

MÓDULO 02: DOMAIN-DRIVEN DESIGN (DDD)

04. Domain Events: Desacoplamiento y Ciclo de Vida

Un **Domain Event** representa un hecho de negocio relevante que ya ocurrió en el pasado (expresado en tiempo pasado, ej: `ToothTreatmentAppliedDomainEvent`). Su objetivo es desacoplar el Agregado principal de efectos secundarios (Facturación, Notificaciones, Inventario).

Despacho mediante EF Core Interceptor

Para asegurar que los eventos solo se publiquen cuando la transacción SQL sea exitosa, utilizamos un `SaveChangesInterceptor` de EF Core acoplado a `MediatR`:

```

public class DispatchDomainEventsInterceptor : SaveChangesInterceptor
{
    private readonly IMediator _mediator;
    public DispatchDomainEventsInterceptor(IMediator mediator) => _mediator = mediator;

    public override async ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result, CancellationToken ct = default)
    {
        var context = eventData.Context;
        if (context == null) return await base.SavingChangesAsync(eventData, result, ct);

        var entities = context.ChangeTracker.Entries<Patient>()
            .Where(e => e.Entity.DomainEvents.Any()).Select(e => e.Entity).ToList();

        var events = entities.SelectMany(e => e.DomainEvents).ToList();
        entities.ForEach(e => e.ClearDomainEvents());

        foreach (var domainEvent in events)
            await _mediator.Publish(domainEvent, ct);

        return await base.SavingChangesAsync(eventData, result, ct);
    }
}

```



30-SECOND SENIOR ENGLISH PITCH

"Domain Events notify external parts of the application when something important happens inside an Aggregate. We collect events inside the Aggregate and dispatch them right during `SaveChangesAsync` using an EF Core Interceptor and MediatR. This keeps the Aggregate decoupled from external handlers like Billing or Notifications."

MÓDULO 03: CQRS PATTERN

05. CQRS: Separación de Lecturas y Escrituras (Read/Write Segregation)

CQRS separa el camino de ejecución de las modificaciones de estado (**Commands**) del camino de consulta de datos (**Queries**).

- **Commands (Write Side):** Pasan por el Modelo de Dominio, Repositorios de Agregado y validan reglas de negocio.
- **Queries (Read Side):** Bypass del Modelo de Dominio y Repositorios. Consultan la base de datos directamente con `.AsNoTracking()` y proyecciones directas a DTOs (o Dapper) para máxima velocidad.

```
// Read Side (Query Handler): Ultra rápido, sin Change Tracker, proyección direct a DTO
public class GetDailyAppointmentsQueryHandler : IRequestHandler<GetDailyAppointmentsQuery, List<DailyAppointmentListDto>>
{
    private readonly ApplicationDbContext _context;

    public GetDailyAppointmentsQueryHandler(ApplicationDbContext context) => _context = context;

    public async Task<List<DailyAppointmentListDto>> Handle(GetDailyAppointmentsQuery query, CancellationToken ct)
    {
        return await _context.Appointments
            .AsNoTracking()
            .Where(a => a.ScheduledTime.Date == query.Date.Date)
            .Select(a => new DailyAppointmentListDto(a.Id, a.Patient.Name.FullName, a.ScheduledTime.Date))
            .ToListAsync(ct);
    }
}
```

30-SECOND SENIOR ENGLISH PITCH

"CQRS separates the write operations from read operations. Commands flow through Rich Domain Models and Aggregates using EF Core with change tracking to enforce business rules. Queries bypass domain models and repositories completely, projecting SQL queries directly to DTOs with AsNoTracking() or Dapper for maximum execution speed."

MÓDULO 03: CQRS PATTERN

06. MediatR Pipeline Behaviors (Cross-Cutting Concerns)

En arquitecturas basadas en CQRS y MediatR, los **Pipeline Behaviors** actúan como un Middleware interno para tus Commands y Queries. Permiten centralizar lógica transversal como **Validación (FluentValidation)**, **Logging de Auditoría**, **Manejo de Excepciones** y **Métricas de Rendimiento** sin ensuciar los Handlers.

Ejemplo: Validation Pipeline Behavior con FluentValidation

```
// Pipeline Behavior Genérico para Validación Transversal
public class ValidationBehavior<TRequest, TResponse> : IPipelineBehavior<TRequest, TResponse>
    where TRequest : IRequest<TResponse>
{
    private readonly IEnumerable<IValidator<TRequest>> _validators;

    public ValidationBehavior(IEnumerable<IValidator<TRequest>> validators)
    {
        _validators = validators;
    }

    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next,
    {
        if (!_validators.Any()) return await next();

        var context = new ValidationContext<TRequest>(request);
        var failures = _validators
            .Select(v => v.Validate(context))
            .SelectMany(result => result.Errors)
            .Where(f => f != null)
            .ToList();

        if (failures.Count != 0)
            throw new ValidationException(failures);

        return await next(); // Continúa al Handler principal solo si la validación es exitosa
    }
}
```



30-SECOND SENIOR ENGLISH PITCH

"MediatR Pipeline Behaviors allow us to implement cross-cutting concerns like logging, performance monitoring, and validation centrally. By placing a ValidationBehavior using FluentValidation inside the MediatR pipeline, all Commands and Queries are automatically validated before hitting their respective Handlers, adhering strictly to the Single Responsibility Principle."